

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)
Department of Computer Science and Engineering
ASSESSMENT TOOL 2 – Scenario Based Assessment

Course Code:	CSA0620	Course Title:	Design and Analysis of Algorithms
CO Assessed:	CO2 – Precision (BL1, BL2, BL3)	Assessment:	Assessment Tool 2 – Scenario Based Assessment
Weightage:	40% of CIA	Total Marks:	100 (2 Questions × 50 Marks)
CO2 Statement:	<i>Determine algorithm efficiency using asymptotic notations and mathematical techniques including Master Theorem and substitution method. (BL1, BL2, BL3)</i>		
Student Name:	Kamalesh T	Reg. No.:	192424228
Section / Batch:	2024	Date:	31/07/2026

Instructions to Students

- Answer BOTH questions. Each question carries 50 Marks. Total: 100 Marks.
- Clearly show all steps in derivations and complexity analysis.
- Use proper asymptotic notation (Big-O, Big-Θ, Big-Ω) wherever required.
- Justify each step in recurrence solving using appropriate methods.
- Neat presentation and logical explanation are mandatory

Question Type	Focus Area	Questions	Marks
Scenario Based Assessment	Apply Master Theorem and asymptotic analysis to real-world scenarios	Q1	100
GRAND TOTAL:		100 Marks	

SECTION A – APPLICATION BASED QUESTIONS

Q1. A hospital management system stores patient data unsorted and retrieves records using Linear

Search during emergencies.

- Explain how Linear Search operates.**
- Analyze its efficiency.**
- Justify whether sorting is beneficial.**

Code of Conduct

I P. Kamalesh T(192424228) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: Kamalesh T _____

RUBRICS FOR CALCULATION

Criteria	Wt.	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Problem Context	20%	Clearly understands the problem and accurately identifies all requirements	Good understanding with minor gaps	Basic understanding; some requirements unclear	Poor understanding or misinterpretation of the problem
Application of Concepts	30%	Accurately applies appropriate concepts to solve complex problems	Applies relevant concepts correctly with minor errors	Applies basic concepts with limited accuracy	Fails to apply appropriate concepts
Problem-Solving Approach	20%	Logical, well-structured, and efficient approach	Mostly logical approach with minor gaps	Basic approach with some inconsistencies	Illogical or unclear approach
Accuracy of Solution	20%	Completely correct solution with proper justification	Mostly correct with minor errors	Partially correct solution	Incorrect or incomplete solution
Clarity	10%	Clear, well-organized, and properly explained solution	Understandable with minor clarity issues	Limited clarity and explanation	Poorly presented and difficult to understand

Marks Evaluation:

Question No.	Understanding of Problem Context (20%)	Application of Concepts (30%)	Problem-Solving Approach (20%)	Accuracy of Solution (20%)	Clarity (10%)	Total Marks
Q1 (100 Marks)						
Overall Total (100)						

Faculty In-charge	Course Coordinator	Head of Department
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

Hospital Management System – Linear Search

Problem Statement

A hospital management system stores patient records in an **unsorted database**. During emergencies, doctors or hospital staff must quickly retrieve a patient's record using **Linear Search**. The task is to explain how Linear Search works, analyze its efficiency, and determine whether sorting the records would improve the searching process.

a) Explain How Linear Search Operates

Definition

Linear Search (also called **Sequential Search**) is one of the simplest searching algorithms. It searches for a required element by comparing the search key with each element in the list, starting from the first element and continuing one by one until the required record is found or the entire list has been examined.

Unlike Binary Search, Linear Search **does not require the data to be sorted**, making it suitable for applications where records are frequently inserted, deleted, or modified.

Working Principle

Suppose the hospital database contains the following patient IDs:

Patient IDs

[1023, 1058, 1015, 1090, 1037, 1062, 1048]

The doctor wants to search for **Patient ID = 1037**.

Step 1

Compare **1023** with **1037**

Not equal → Move to next record.

Step 2

Compare **1058** with **1037**

Not equal → Move to next record.

Step 3

Compare **1015** with **1037**

Not equal → Continue searching.

Step 4

Compare **1090** with **1037**

Not equal → Continue searching.

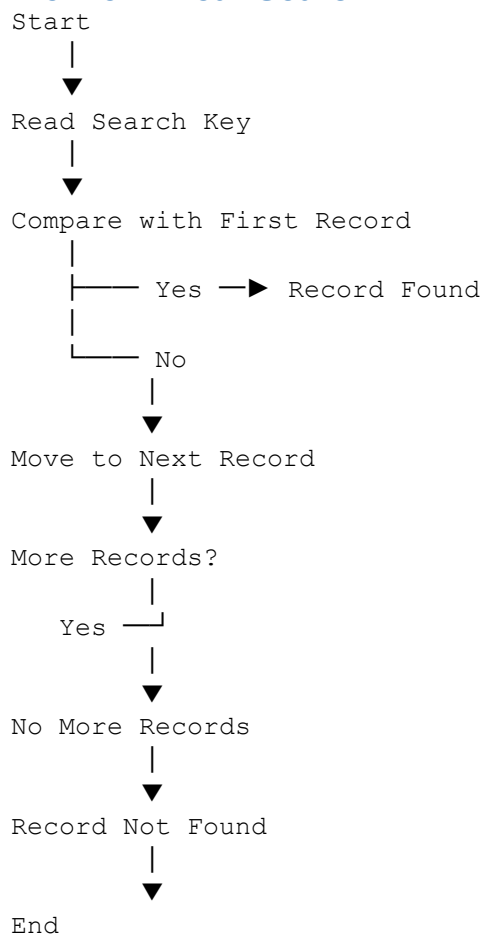
Step 5

Compare **1037** with **1037**

Match found.

Return the patient's record.

Flow of Linear Search



Algorithm

1. Start searching from the first patient record.

2. Compare the current record with the search key.
3. If both are equal, return the patient record.
4. Otherwise, move to the next record.
5. Repeat until the record is found or the last record is reached.
6. If no match is found, display **"Patient Record Not Found."**

b) Analyze Its Efficiency

The efficiency of Linear Search depends on the position of the required patient record.

Best Case

The patient record is present in the **first position**.

Example

Search Key = 1023

1023 ✓

Only **one comparison** is required.

Time Complexity

$O(1)$

Average Case

The patient record is located somewhere in the middle.

Example

1023
1058
1015
1090
1037 ✓

Approximately **$n/2$ comparisons** are required.

Time Complexity

$O(n)$

Worst Case

The patient record is located at the last position or is not available.

Example

```
1023
1058
1015
1090
1037
1062
1048
Search = 1100
```

Every record must be checked.

Total comparisons = **n**

Time Complexity

$O(n)$

Space Complexity

Linear Search uses only a few variables.

No additional data structure is required.

Therefore,

Space Complexity = $O(1)$

Comparison Count Example

For **7 patient records**

Best Case

Comparisons = **1**

Average Case

Comparisons $\approx$ **4**

Worst Case

Comparisons = **7**

Advantages

- Very simple to understand and implement.
- Works efficiently with small datasets.
- Does not require sorted data.

- Suitable when records are continuously inserted or deleted.
- Requires very little memory.
- Can search any type of data structure.

Disadvantages

- Slow for large datasets.
- Every record may need to be examined.
- Search time increases linearly with the number of records.
- Not suitable for applications with millions of records where rapid searching is essential.

c) Justify Whether Sorting is Beneficial

Sorting organizes patient records in ascending or descending order based on a key such as Patient ID.

Once the records are sorted, the hospital can use **Binary Search**, which is much faster than Linear Search.

Advantages of Sorting

- Enables Binary Search.
- Reduces search time significantly.
- Improves efficiency for large databases.
- Faster retrieval during repeated searches.
- Better performance when records rarely change.

Disadvantages of Sorting

- Sorting requires additional processing time.
- Every new patient record must be inserted in the correct position.
- Updating patient details may require re-sorting.
- Frequent insertions and deletions reduce the benefit of keeping data sorted.

Comparison Between Linear Search and Binary Search

Feature	Linear Search	Binary Search
Data Requirement	Unsorted or Sorted	Sorted Only
Best Case	$O(1)$	$O(1)$
Average Case	$O(n)$	$O(\log n)$
Worst Case	$O(n)$	$O(\log n)$
Implementation	Very Easy	Moderate
Suitable for Dynamic Data	Yes	Less Suitable
Suitable for Large Databases	No	Yes

Hospital Scenario Analysis

Emergency Situation

If patient records are updated continuously:

- New admissions occur every minute.
- Patient discharge records are removed.
- Medical information is modified frequently.

Maintaining sorted data becomes expensive.

In such situations, **Linear Search** is practical because it works directly on unsorted records.

Large Hospital Database

Suppose a hospital stores **1,000,000 patient records**.

- Linear Search may require checking nearly **1,000,000 records** in the worst case.
- Binary Search requires only about **20 comparisons** because:

$$\log_2(1,000,000) \approx 20 \quad \log_2(1,000,000) \approx 20$$

Thus, sorting becomes highly beneficial when searches are performed frequently.

Result

Linear Search examines patient records one by one until the required record is found. It is simple, requires no sorting, and uses constant memory. However, its running time is **$O(n)$** , making it inefficient for very large hospital databases.

Sorting patient records enables Binary Search, reducing the search time to **$O(\log n)$** . Therefore, sorting is beneficial when the database is large and searches are frequent. If records are updated continuously, maintaining a sorted order may not be worthwhile, and Linear Search remains a practical solution.

Conclusion

Linear Search is an effective searching technique for **small or frequently changing hospital databases** because it works directly on unsorted data and is easy to implement. However, as the number of patient records increases, its performance degrades due to its **$O(n)$** time complexity. For large hospitals with frequent search operations and relatively stable data, sorting the records and using **Binary Search** provides much faster retrieval, making it the preferred approach.